

Lecture 4

EE 421 / CS 425

Digital System Design

Fall 2026

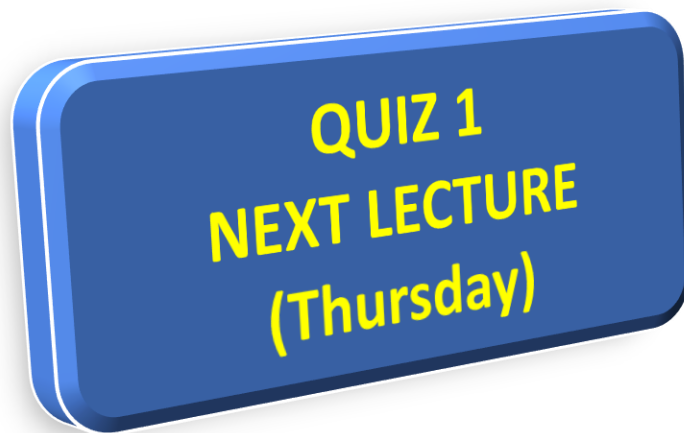
Shahid Masud

Topics

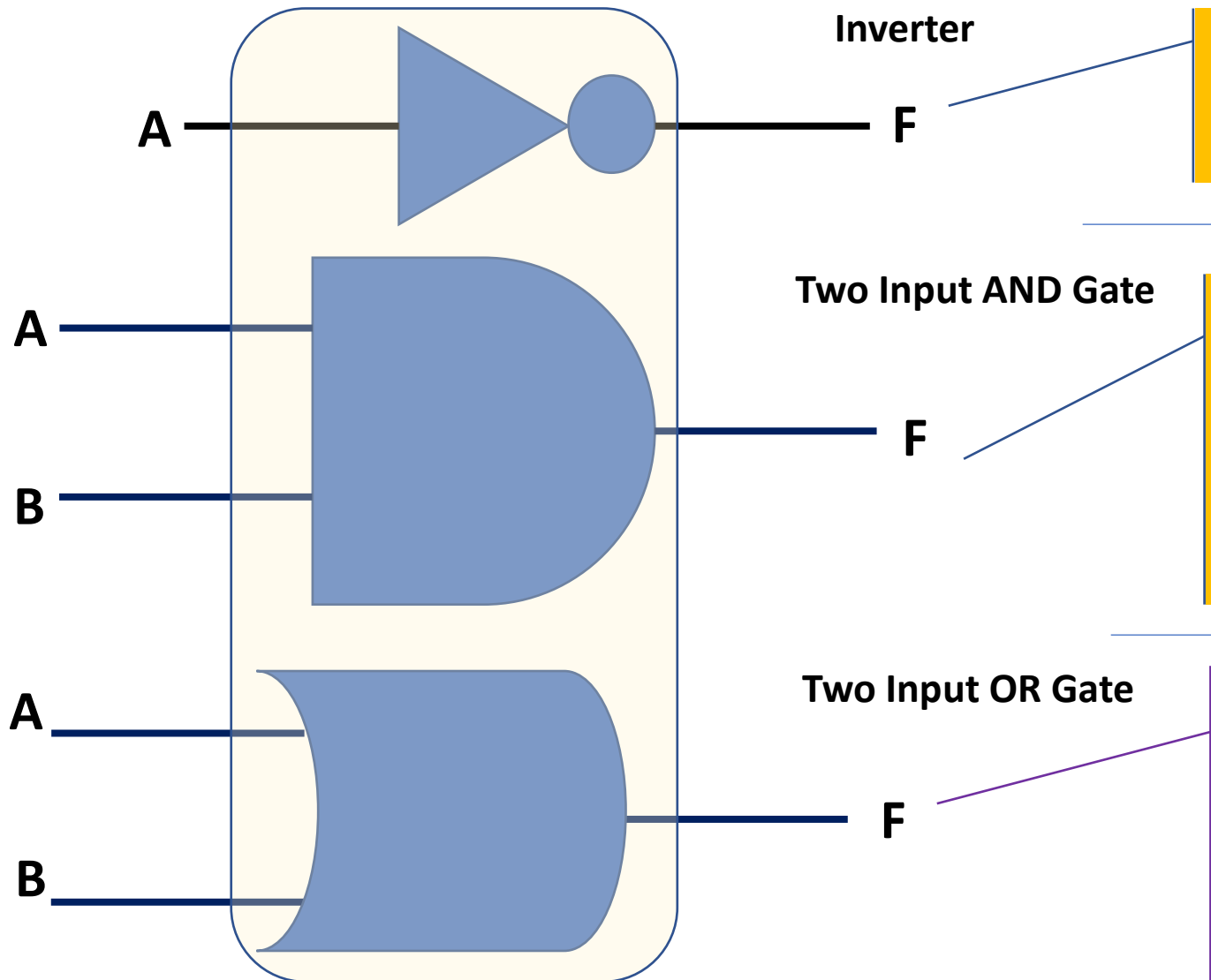
Some examples from WinLogiLab (from last time)

FROM BEHAVIOUR TO IMPLEMENTATION

- Connecting logic expression to physical circuits: Boolean Algebra, K-Maps to logic gates, XOR
- Combinational Logic Implementation using functional mapping to Decoders, Memory and Multiplexers
- Implementation Constraints
- -----
- Introduction to Timing Characteristics



Gates (Primary Gates)



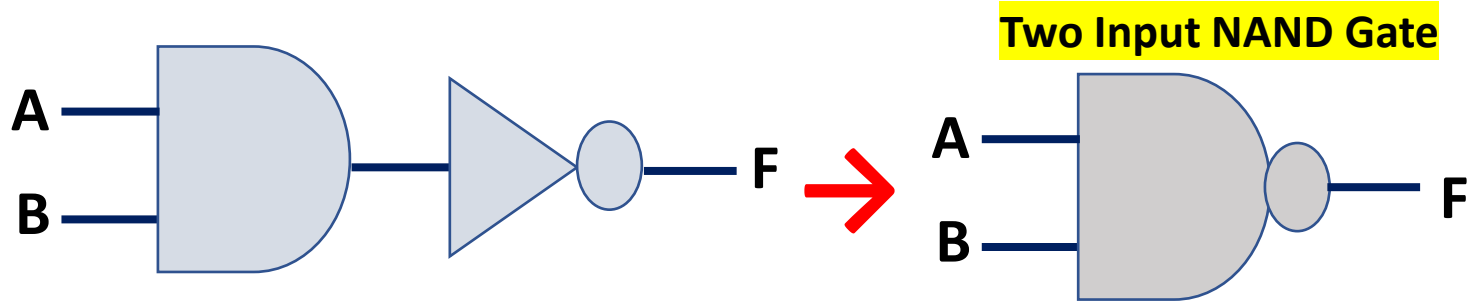
BEHAVIOURAL MODELS

A	F
0	1
1	0

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

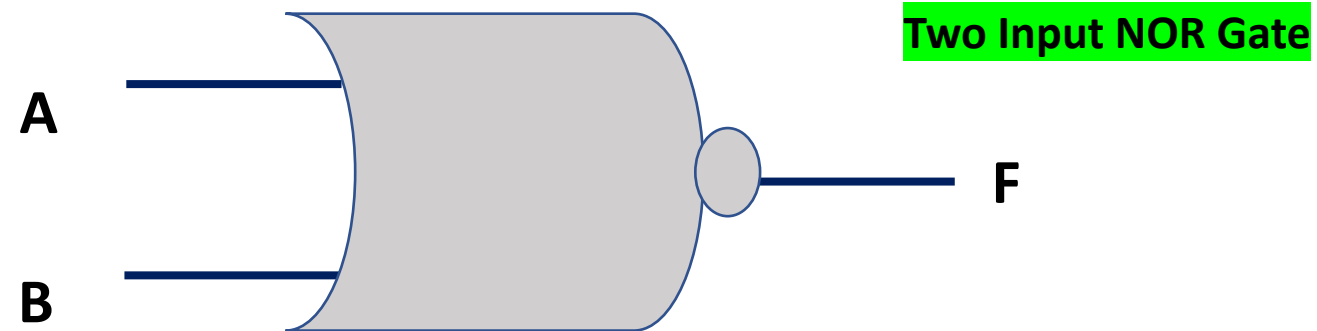
A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

Gates (Compound Gates)



AND Gate followed by Inverter

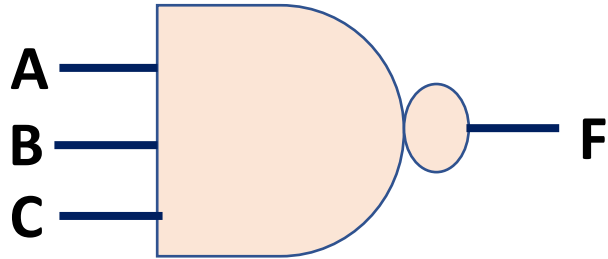
A	B	F
0	0	1
0	1	1
1	0	1
1	1	0



OR Gate followed by Inverter

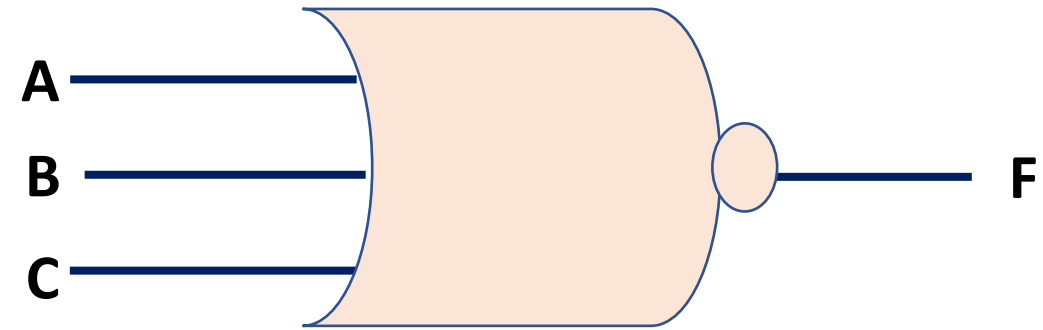
A	B	F
0	0	1
0	1	0
1	0	0
1	1	0

3 Input Compound Gates



Three Input NAND Gate

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

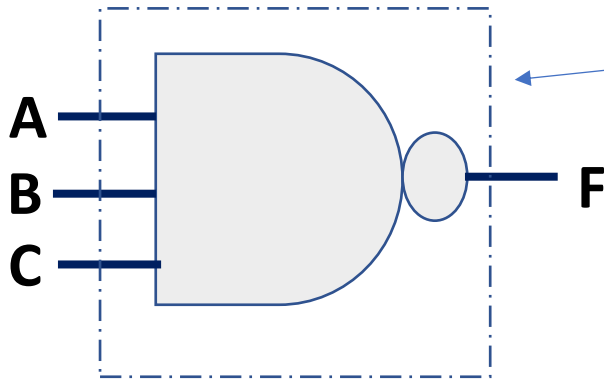


Three Input NOR Gate

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

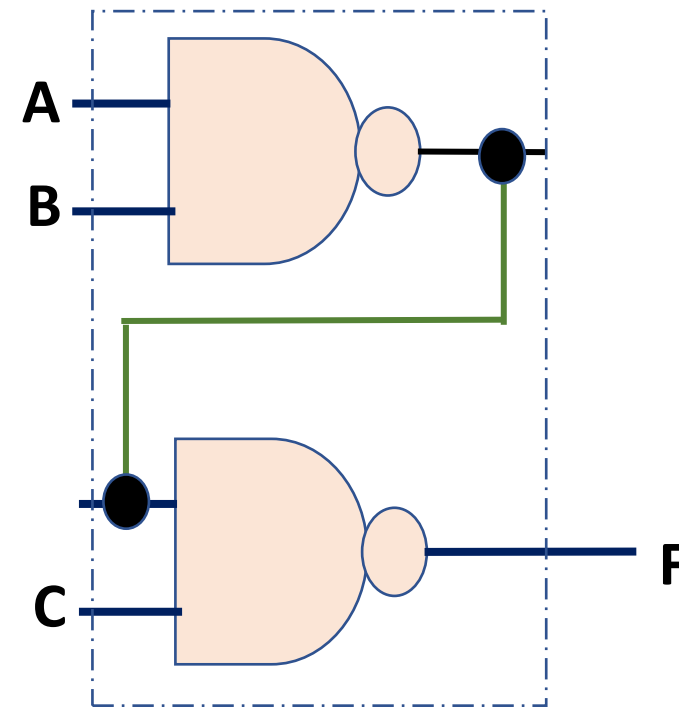
A Question about Compound Gates

Is this 3 input Nand gate same as the configuration on the right?



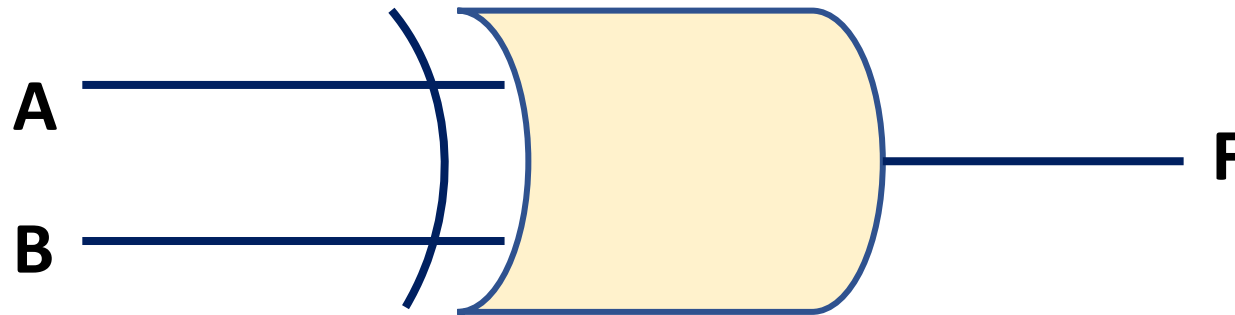
Are these equivalent Circuits?

Or Not Equivalent??



Complex Gates – the XOR and XNOR

Two Input Exclusive OR Gate



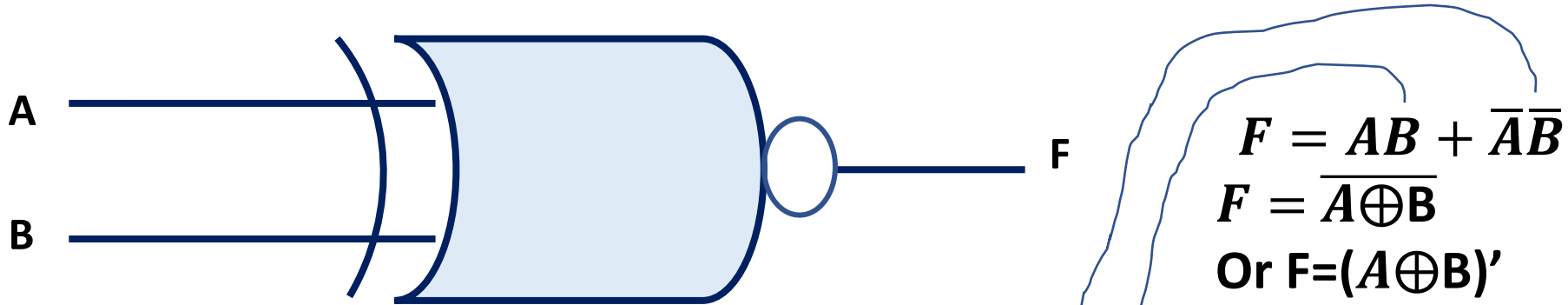
$$F = \bar{A}B + A\bar{B}$$

$$F = A \oplus B$$

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

The XNOR Complex gate

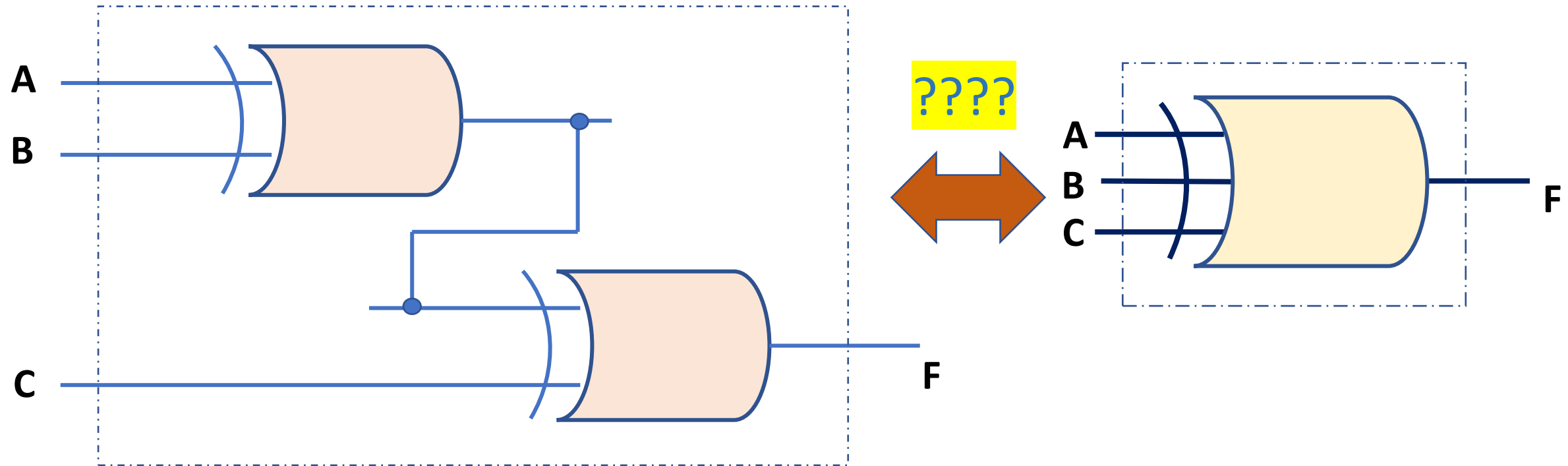
Two Input Exclusive NOR Gate



A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

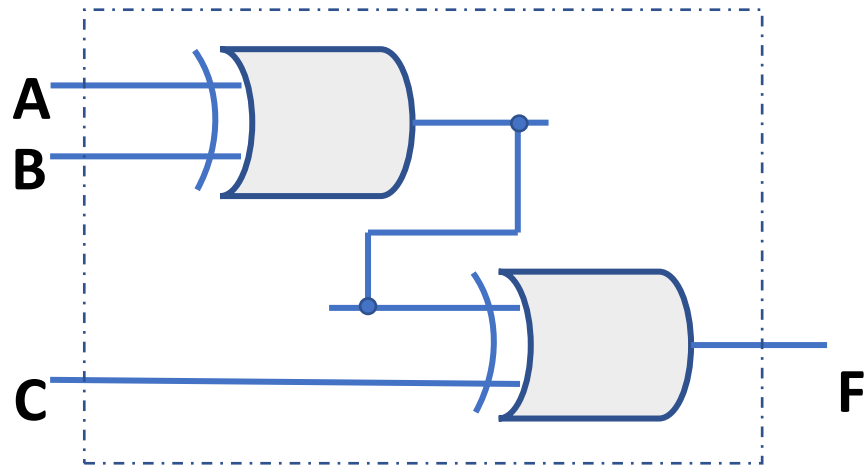
Question?

Are these two gates equivalent, Yes? Or No?



$$F = A \oplus B \oplus C$$

Three Input XOR Gate



$$F = A \oplus B \oplus C$$

A	B	C	$(A \oplus B)$	$(A \oplus B) \oplus (C)$	F
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	0	0
1	1	0	0	0	0
1	1	1	0	1	1

K Map of $(A \oplus B) \oplus (C)$

C, AB	00	01	11	10
0	0	1	0	1
1	1	0	1	0

This is the right way to work with XOR Gates

(but not the NAND or NOR gates)

Functional Digital Circuits – MSI (Medium Scale Integration)

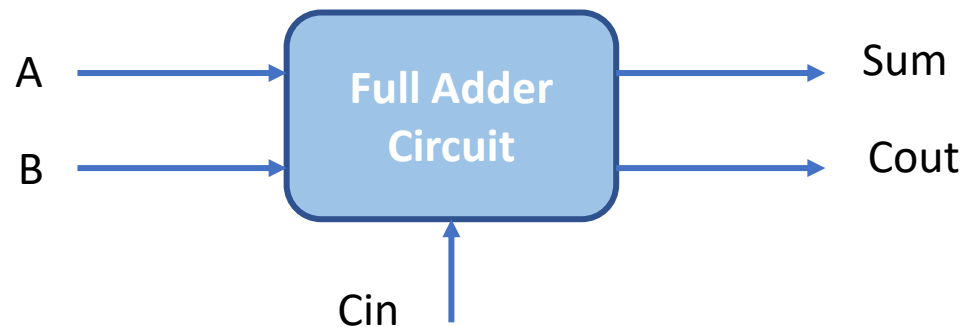
Circuits that are formed from simple, compound or complex gates

Examples:

- Adder/Subtractor circuits
- Decoders
- Multiplexers
- Parity Checkers
- Memory Elements
- Counters, Modules
- SIPO, PISO, or FIFO, LIFO, – Require **REGISTERS**
- Etc.

Direct Implementation of
Complex Logic Functions

1 Bit Full Adder Circuit



Truth table represents **behaviour** of inputs and outputs

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

K-Map for Sum Function

Cin, AB →	00	01	11	10
0	0	1	0	1
1	1	0	1	0

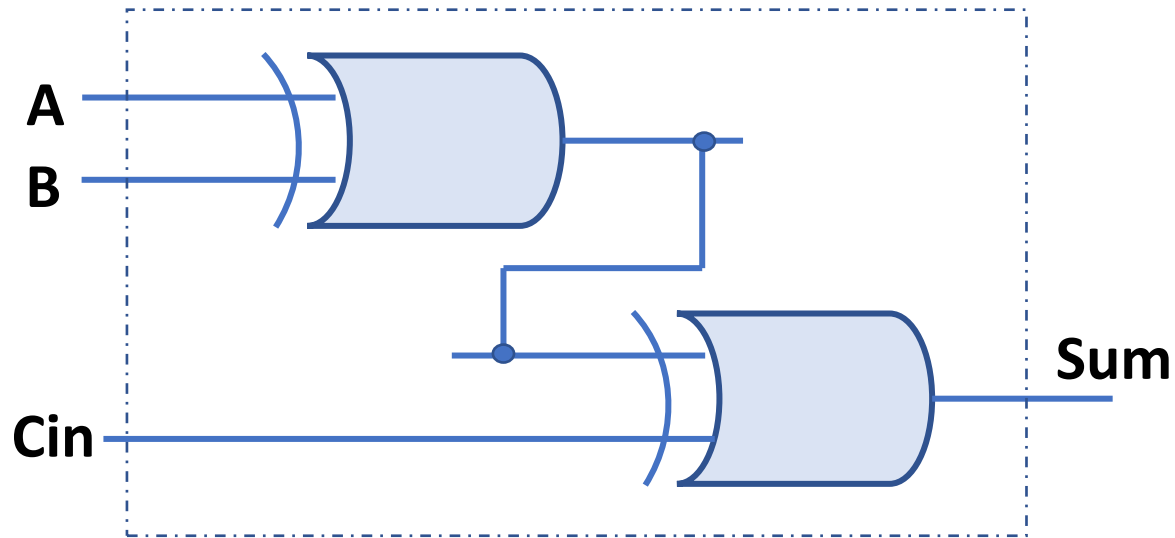
$$Sum = A \oplus B \oplus Cin$$

K-Map for Cout Function

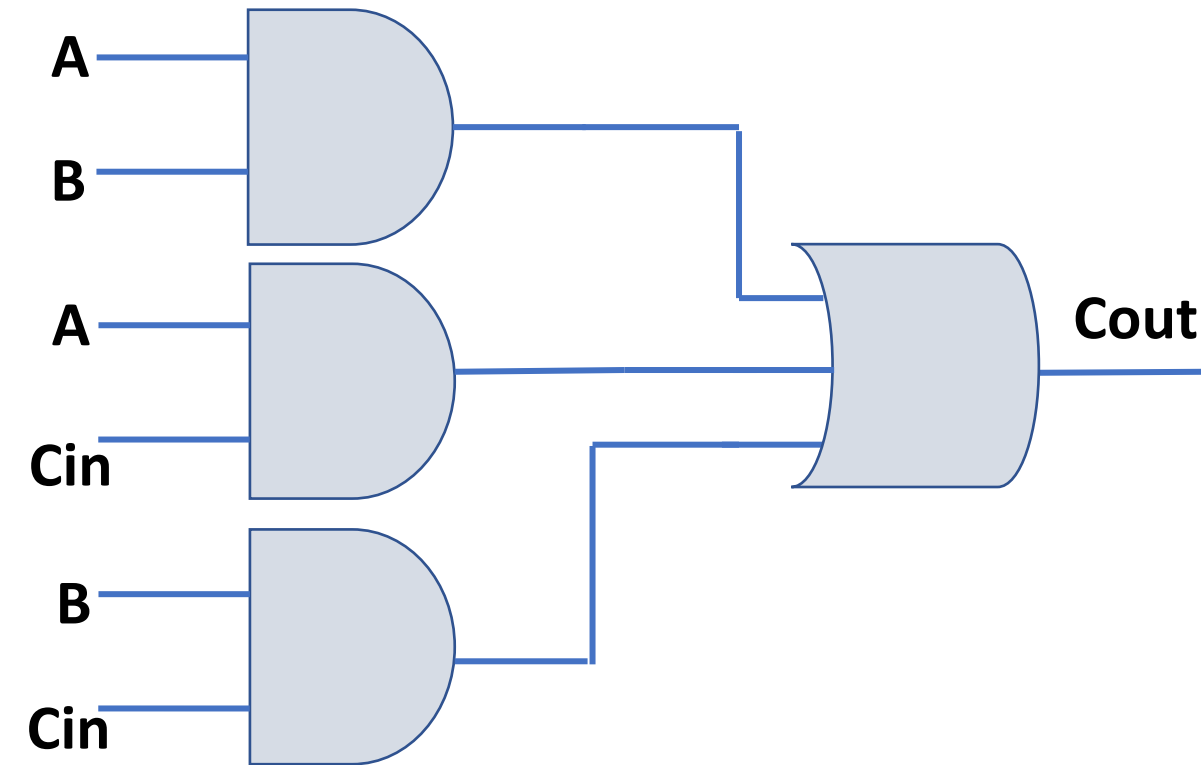
Cin, AB →	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$Cout = AB + ACin + BCin$$

Full Adder Implementation using Logic Gates

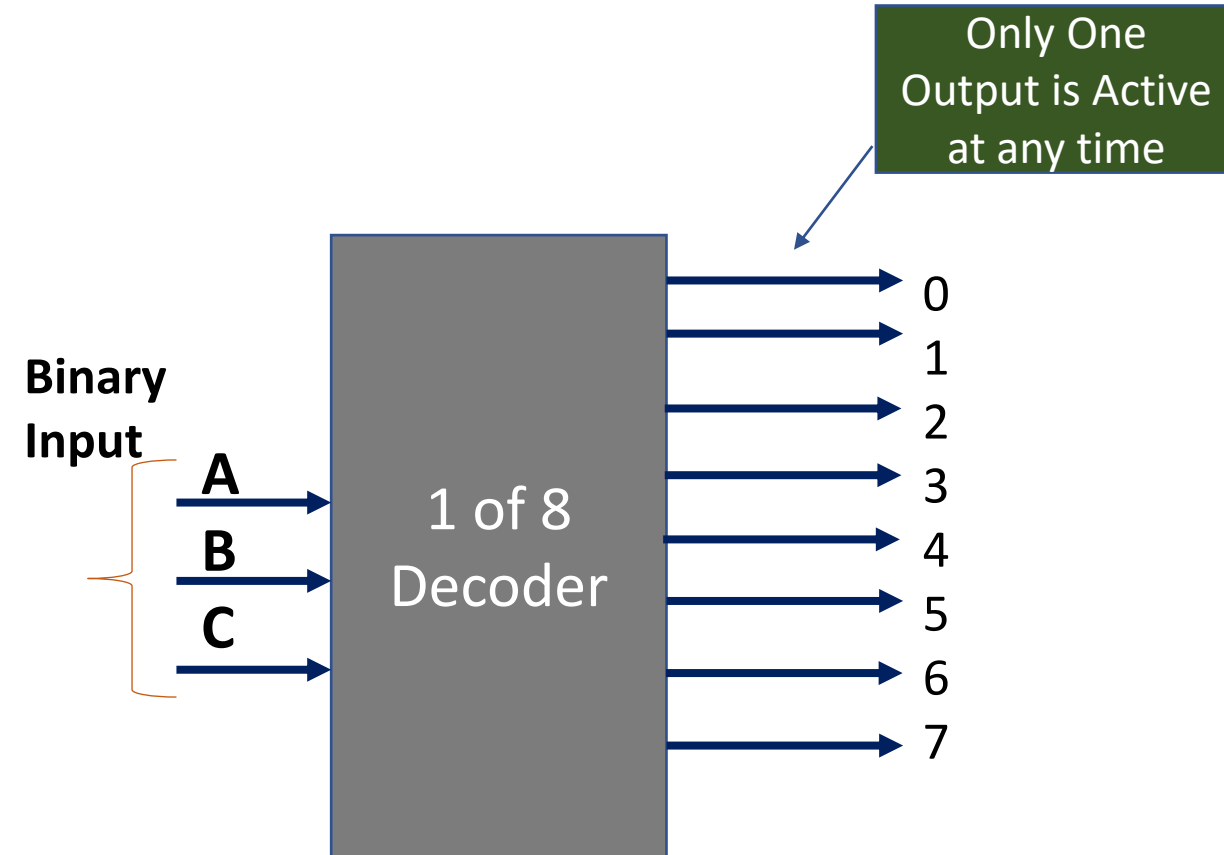
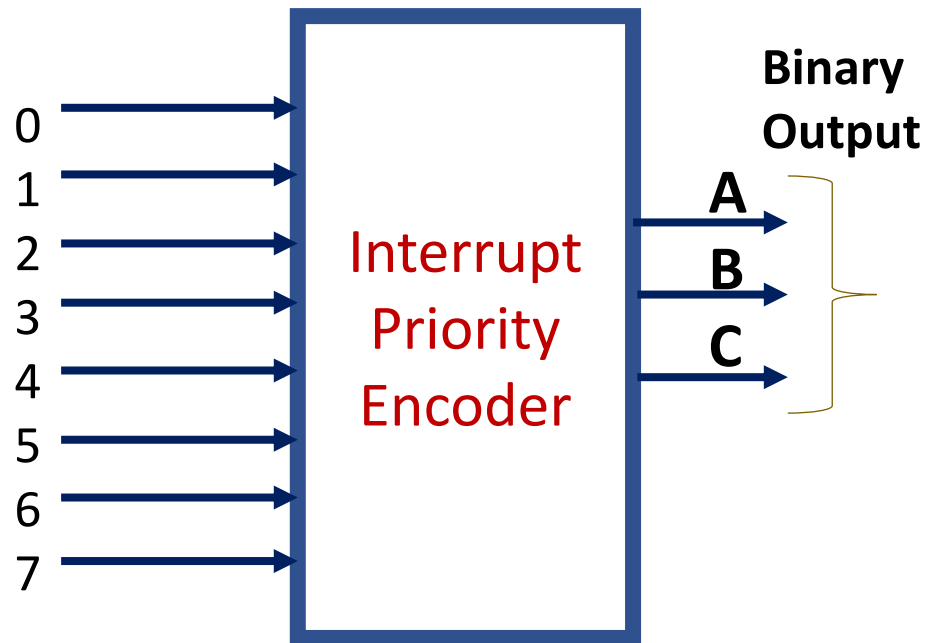


$$Sum = A \oplus B \oplus Cin$$

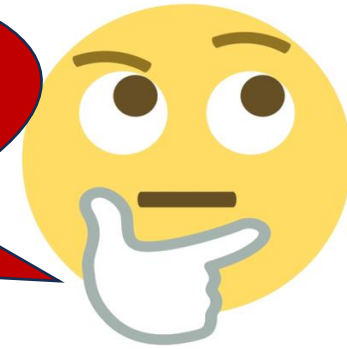


$$Cout = AB + ACin + BCin$$

Encoders and Decoders Functional Chips



Remember: We are dealing
with 'Complex' and 'High
Speed Circuits'



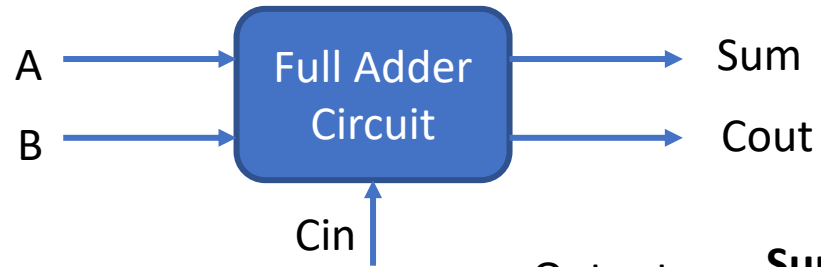
Logic Implementation vs Functional Mapping

Low Level Gates

High Level Functional Modules

Design Complexity

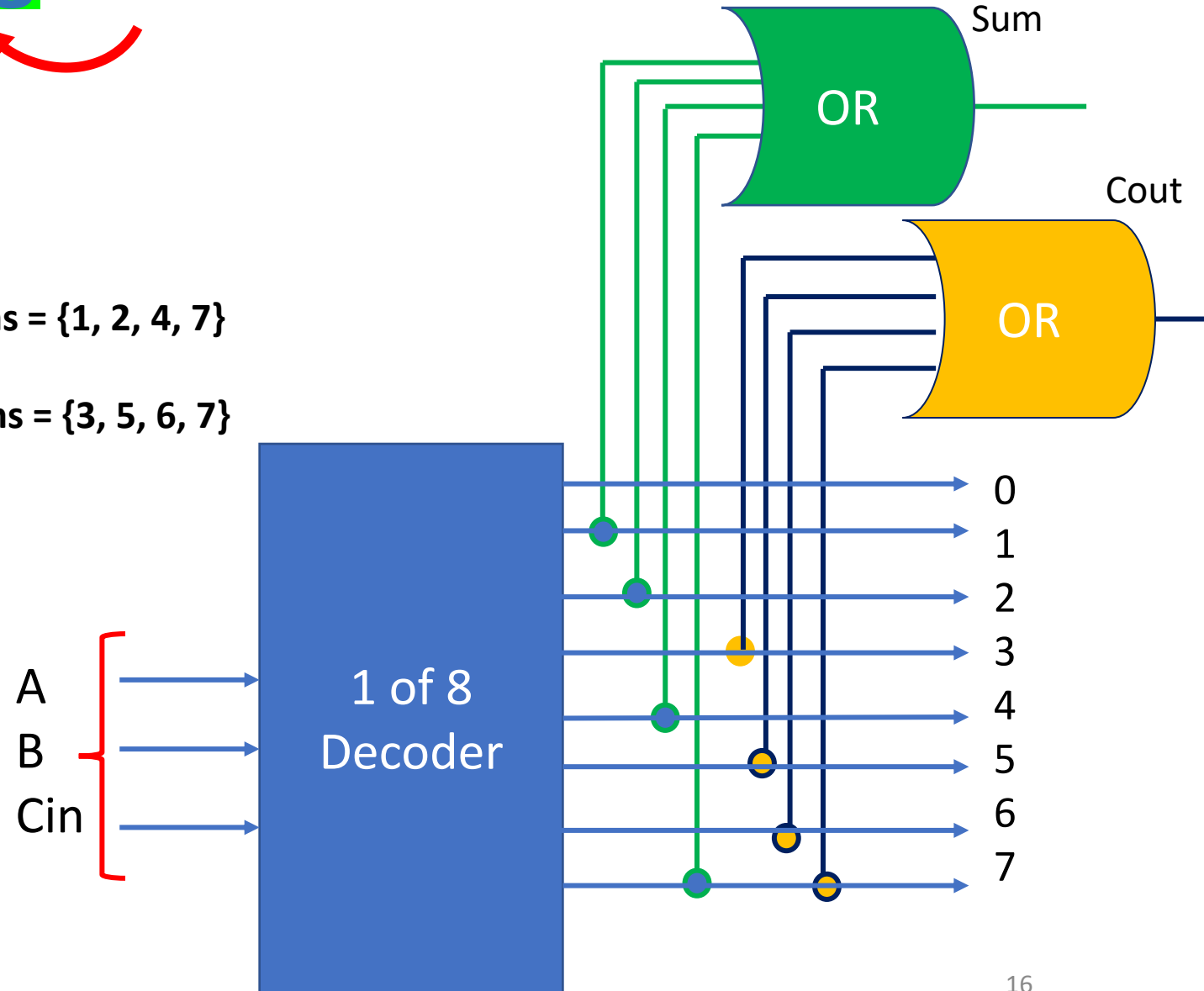
Full Adder Mapping on Decoder



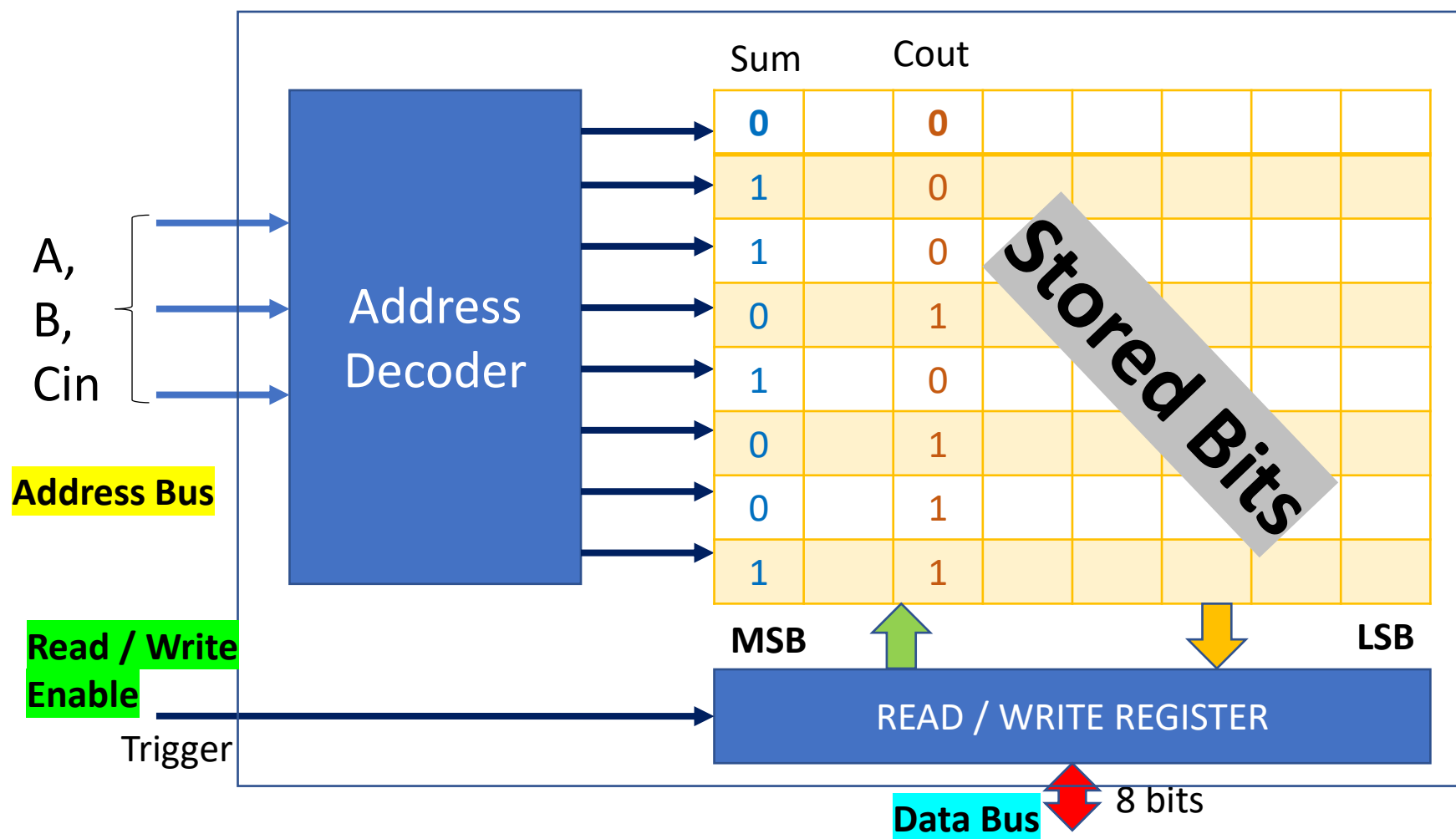
Inputs			Outputs	
A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sum Minterms = {1, 2, 4, 7}

Cout Minterms = {3, 5, 6, 7}



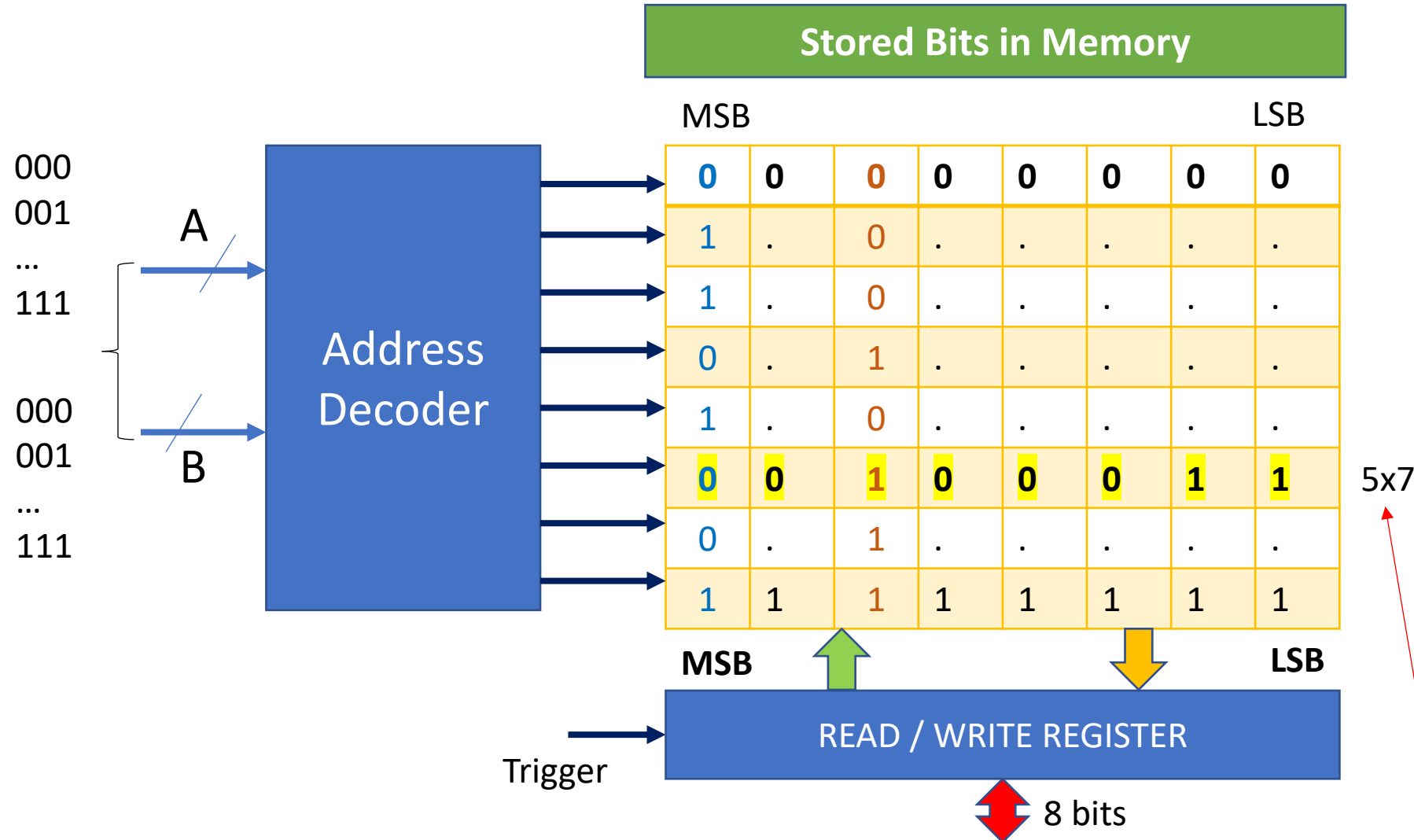
Memory Storage (RAM or ROM) as a functional logic block – **Lookup Tables**



Truth Table Outputs are Programmed in the memory Lookup Table

MSB output (Bit 7) is Sum
MSB-2 output (Bit 5) is Cout

Implement 3 bit x 3 bit Multiplier using Memory

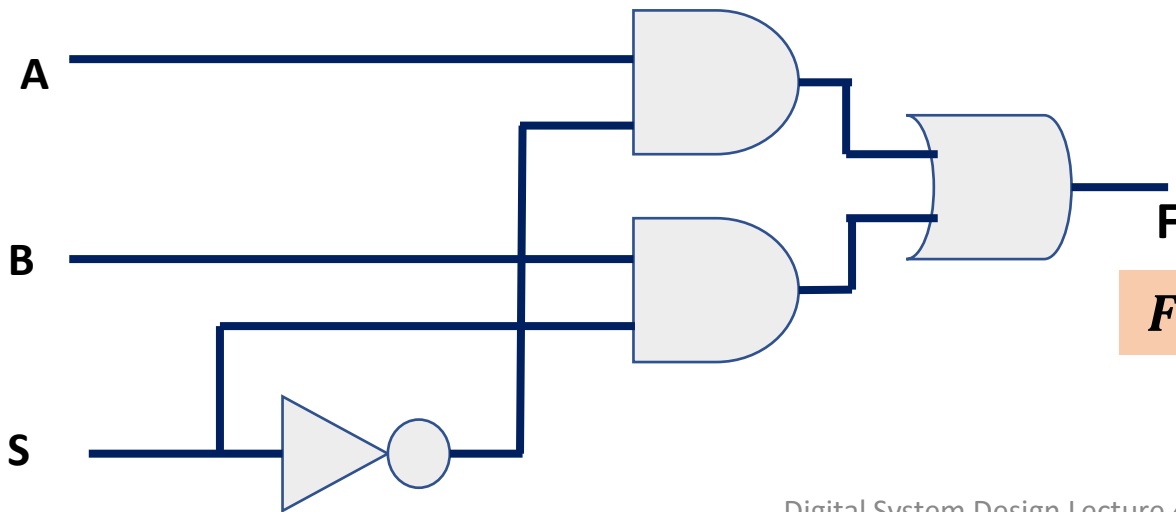
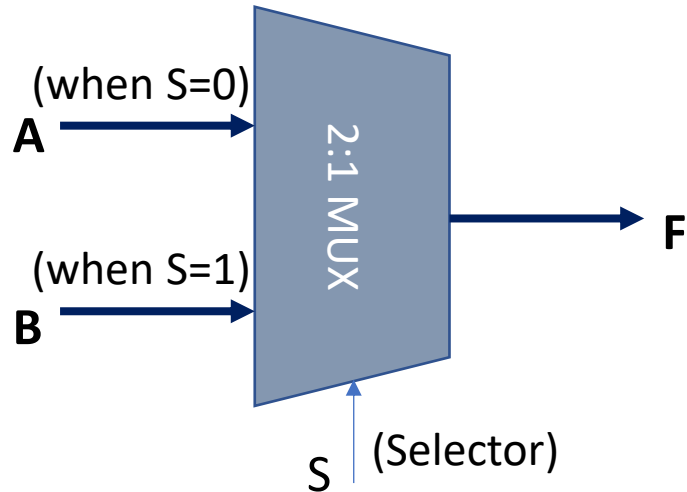


Truth Table Outputs are Programmed in the memory Lookup Table

A	B	AxB	Data
0	0	0	00 H
0	1	0	00 H
...	...	...	...
0	7	0	00 H
0	1	0	00 H
1	1	1	01 H
2	1	2	02 H
...	...	...	...
3	2	6	06 H
...	...	...	...
5	7	35	23 H
...	...	...	...

Multiplexer MUX Functional Module

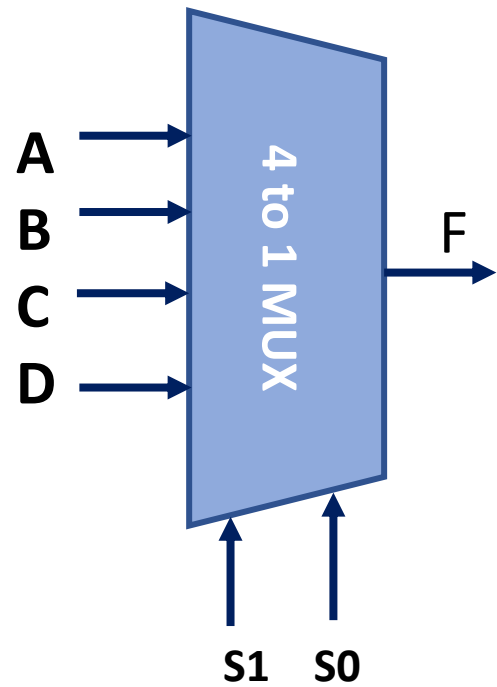
Selects one of many inputs at the output



$$F = \bar{S}A + SB$$

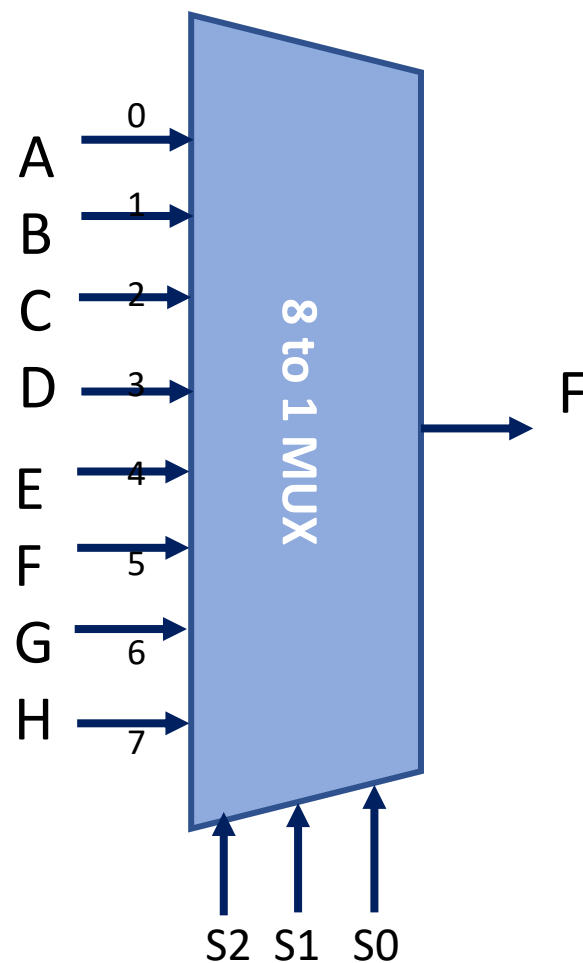
Inputs		Select	Output
A	B	S	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

4 to 1 MUX



Inputs				Select		Output
A	B	C	D	S1	S0	F
A	B	C	D	0	0	A
A	B	C	D	0	1	B
A	B	C	D	1	0	C
A	B	C	D	1	1	D

8 to 1 MUX



Input Lines								Select lines			Output
A	B	C	D	E	F	G	H	S2	S1	S0	F
A	B	C	D	E	F	G	H	0	0	0	A
A	B	C	D	E	F	G	H	0	0	1	B
A	B	C	D	E	F	G	H	0	1	0	C
A	B	C	D	E	F	G	H	0	1	1	D
A	B	C	D	E	F	G	H	1	0	0	E
A	B	C	D	E	F	G	H	1	0	1	F
A	B	C	D	E	F	G	H	1	1	0	G
A	B	C	D	E	F	G	H	1	1	1	H

Map a full adder to a 4 to 1 MUX

Full Adder Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

MUX Selector Groups (2 Bits, 4 groups)

Select = 00

Observations

Sum output follows Cin; Cout remains 0

Select = 01

Sum output follows Cin'; Cout follows Cin

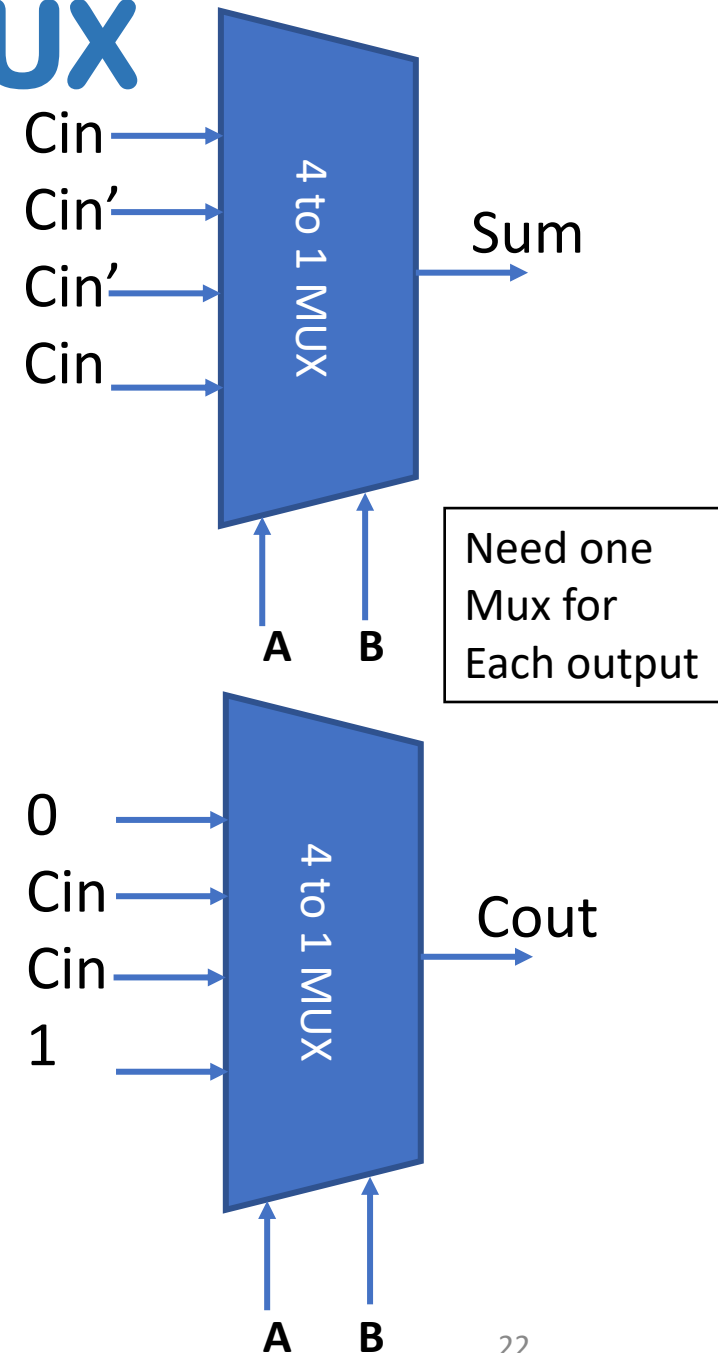
Select = 10

Sum output follows Cin'; Cout follows Cin

Select = 11

Sum output follows Cin; Cout remains 1

AND
Selector is same as inputs
AB



Mapping 16 Minterms to a 8 to 1 MUX

Output

A	B	C	D	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

3 Select Lines so
8 groups

Observations

F follows D

F follows D

F follows D'

F remains 0

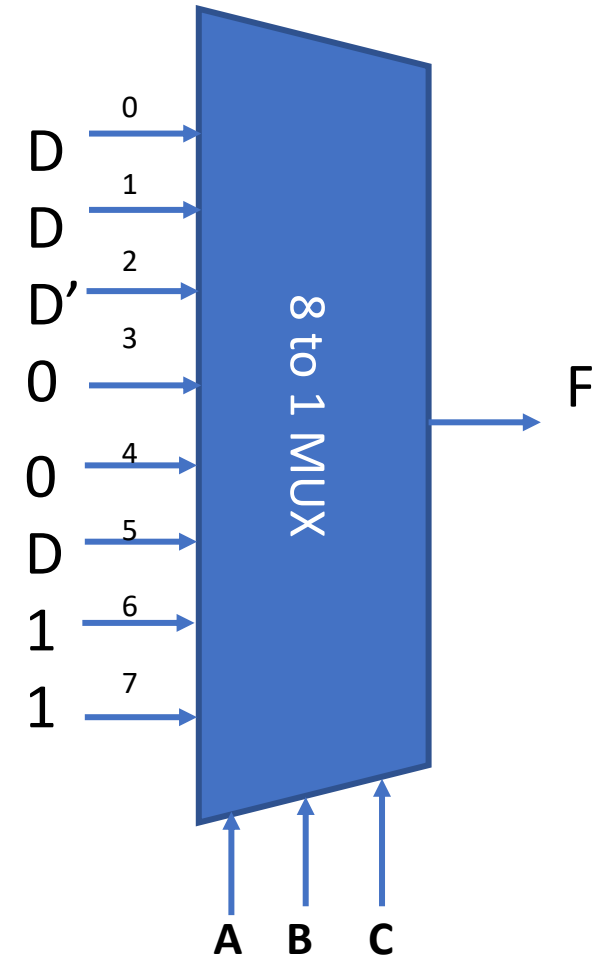
F remains 0

F follows D

F remains 1

F remains 1

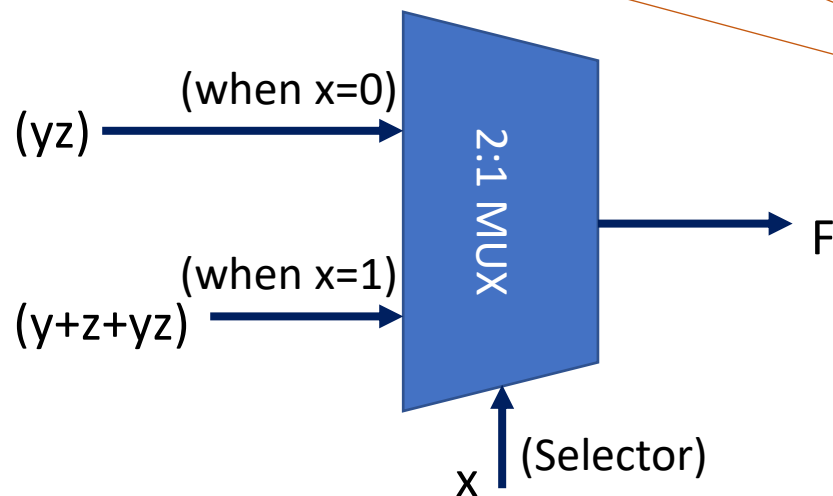
Mapping



Minterms {1, 3, 4, 11, 12, 13, 14, 15}

Example: Implement logic functions using only 2 to 1 MUXes

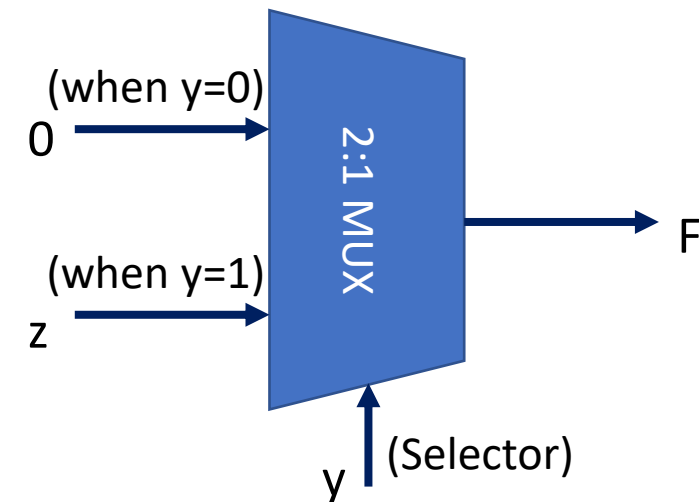
$F = x'(yz) + x(y+z+yz)$, use x as selector input



Now we have to find a way to map (yz) and $(y+z+yz)$ into 2 to 1 MUX

Use y as select lines

For (yz) ; when $y = 0$, output is 0
when $y = 1$, output is z



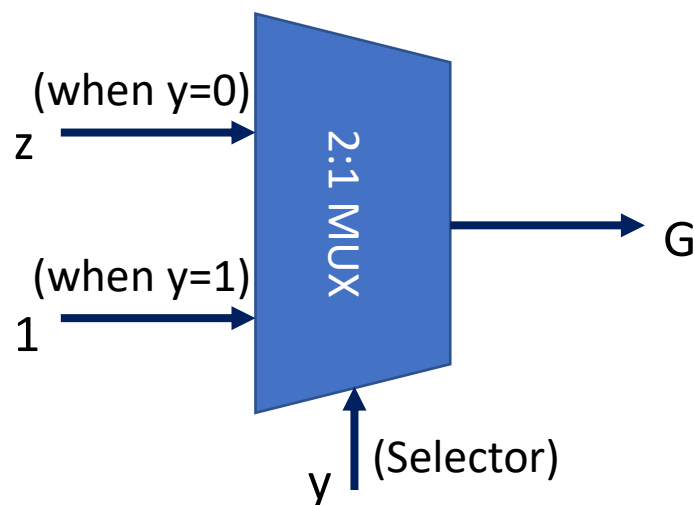
Contd - Mapping only on 2 to 1 MUX

Let $G = (y+z+yz)$

Then $G = y + z(y+y') + yz$

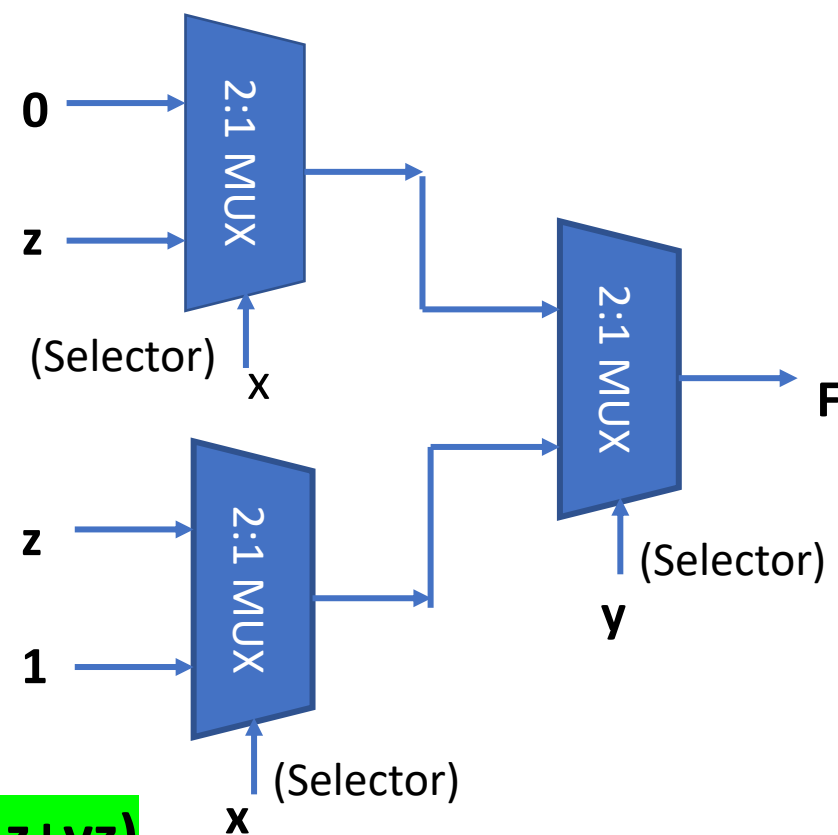
$G = y(1+z+y) + z(y')$

$G = y(1) + y'(z)$



Implementation of $F = x'(yz) + x(y+z+yz)$

Full implementation using only 2 to 1 MUX



Conclusion

- Behaviour of Digital System is described in Truth Table or Boolean Algebra
- For Implementation of this Behaviour, we can use:
 - Logic Gates – detailed and elaborate gate level design - Complicated
 - Decoders
 - Multiplexers
 - Memory

High Level Mapping to
Manage COMPLEXITY

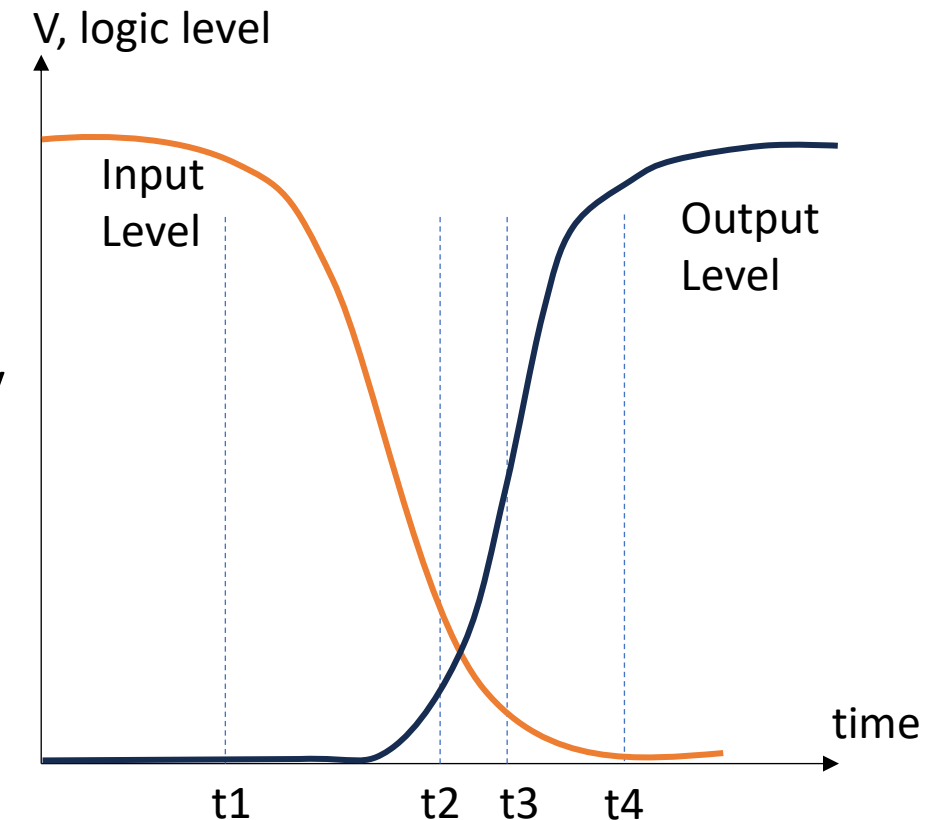
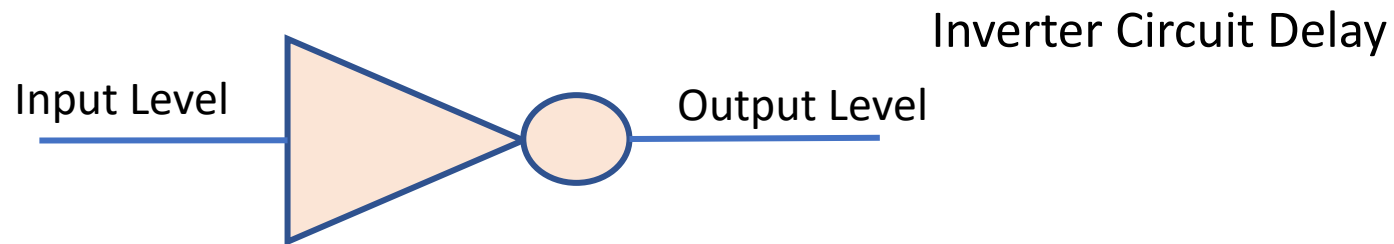
Topics

- Pulse Circuits
- Timing Related Effects in Combinational Circuits
- Glitches
- Hazards
- Identify and Remove Glitches and Hazards
- If we have time, we may start
 - Review of Sequential Digital Logic Circuits
 - Basics of Latches
 - Basics of Flipflops
- QUIZ 1 NEXT LECTURE

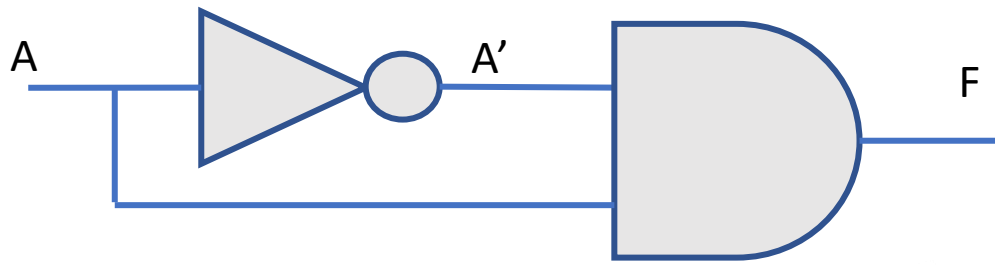
Timing Waveforms

Every physical circuit has some processing delay; once the input has settled, **the correct output appears after some time.**

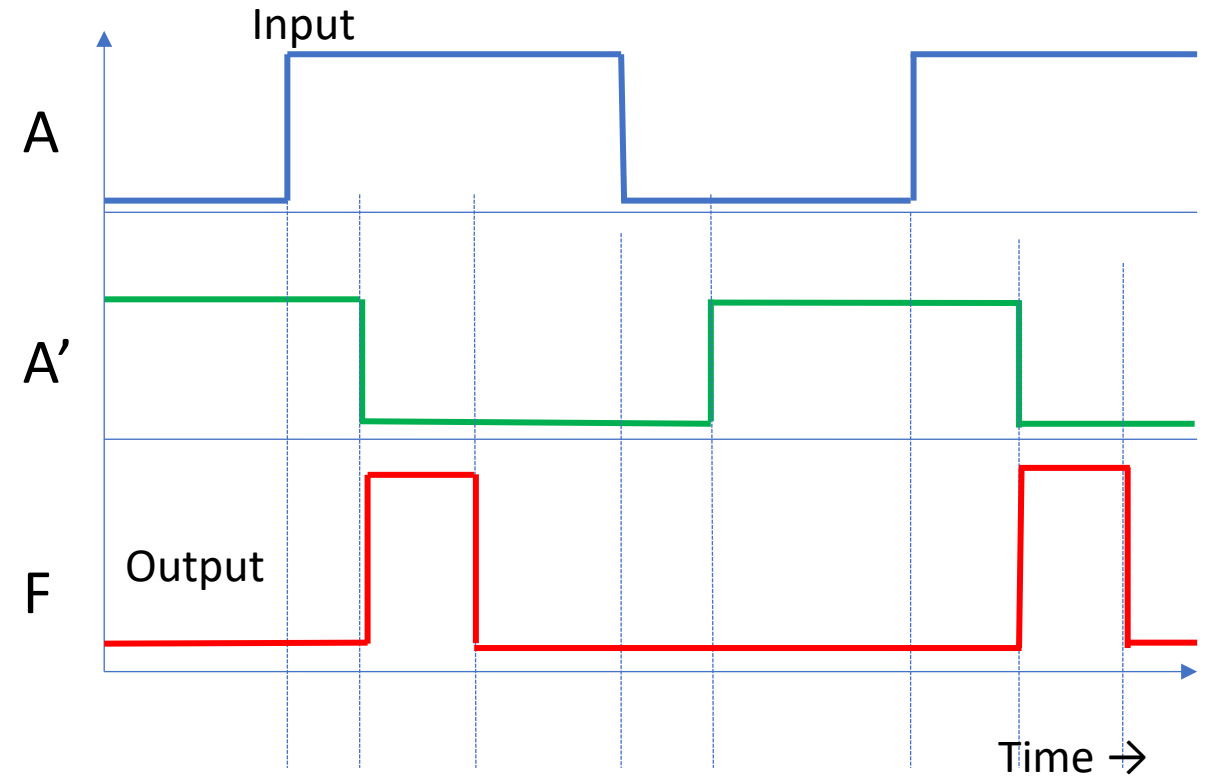
Propagation delay from a gate is defined by:
 t_{plh} = time delay in output going from low to high
 t_{phl} = time delay in output going from high to low



A simple pulse circuit



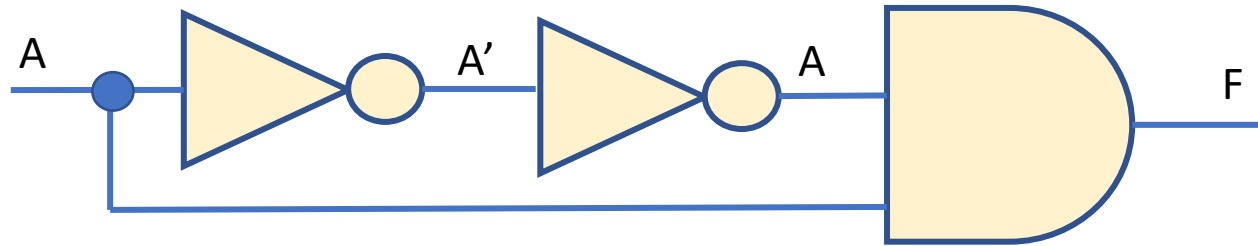
Each gate has Propagation Delay



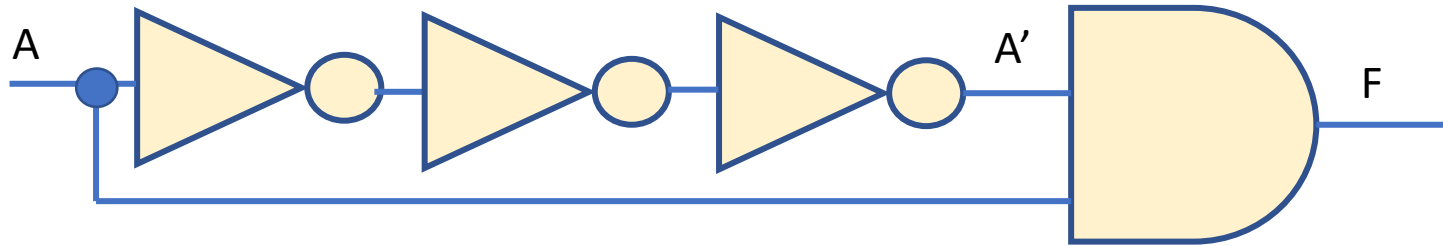
Question: What will be the Output if there are two inverters in series?

Question: What will be the Output if there are three inverters in series?

Two and Three Inverters in simple pulse circuit

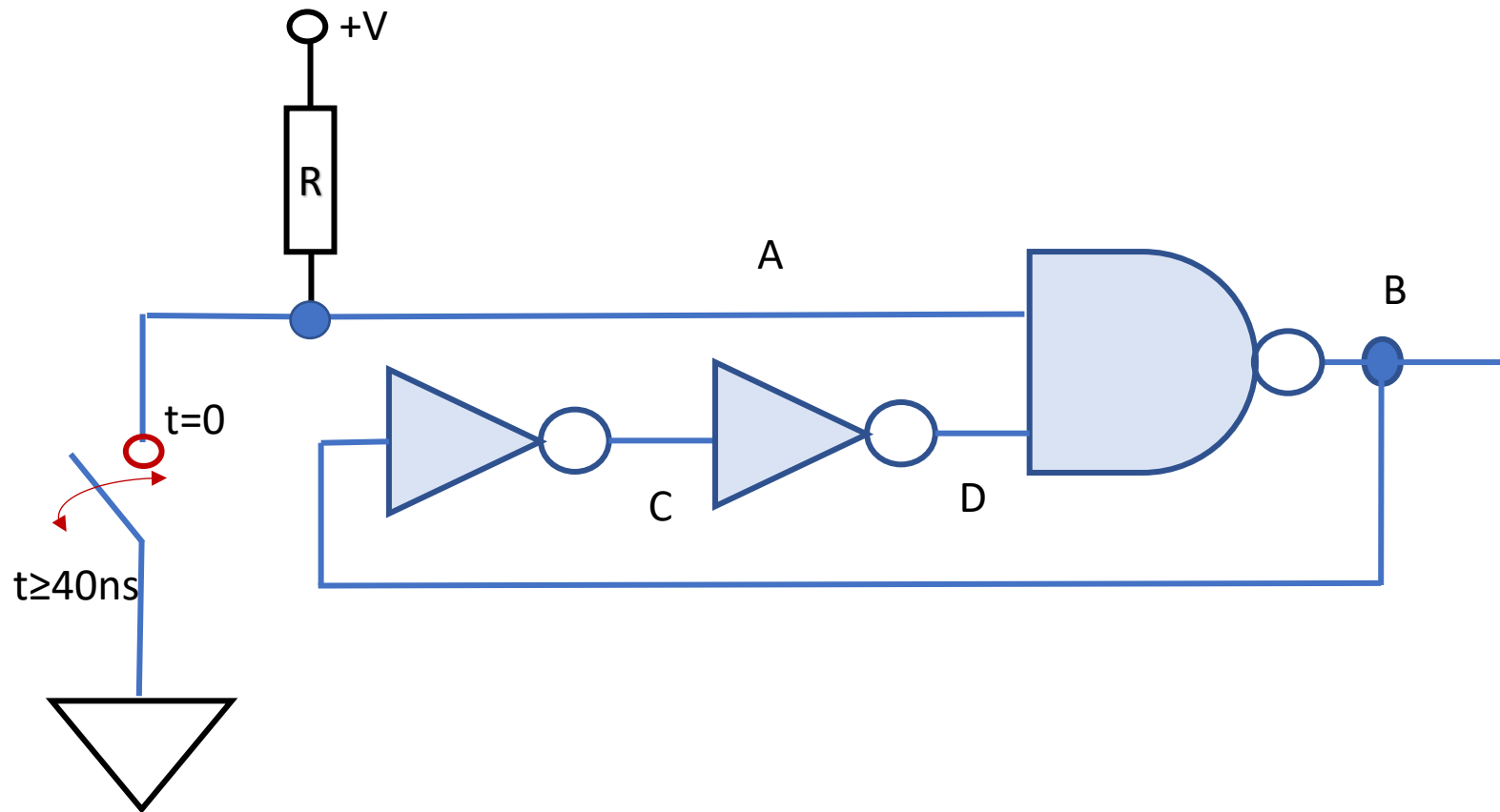


For even number of Inverters:
Check input A transitions from
0 to 1 and from 1 to 0
Pulse behaviour is not observed
(Hint: Check whether F is like A?)



For Odd number of Inverters:
At 0 to 1 transitions at A, a pulse will
appear at F. The pulse width is proportional
to number of inverters as their propagation
delay adds up

A Pulse Shaper Circuit



Case 1:

Assume propagation delay of gates as 10ns

Switch is closed at $t=0\text{ns}$, thus:

A goes to '0' at time $t=0\text{ns}$

B goes to '1' at time $t=10\text{ns}$

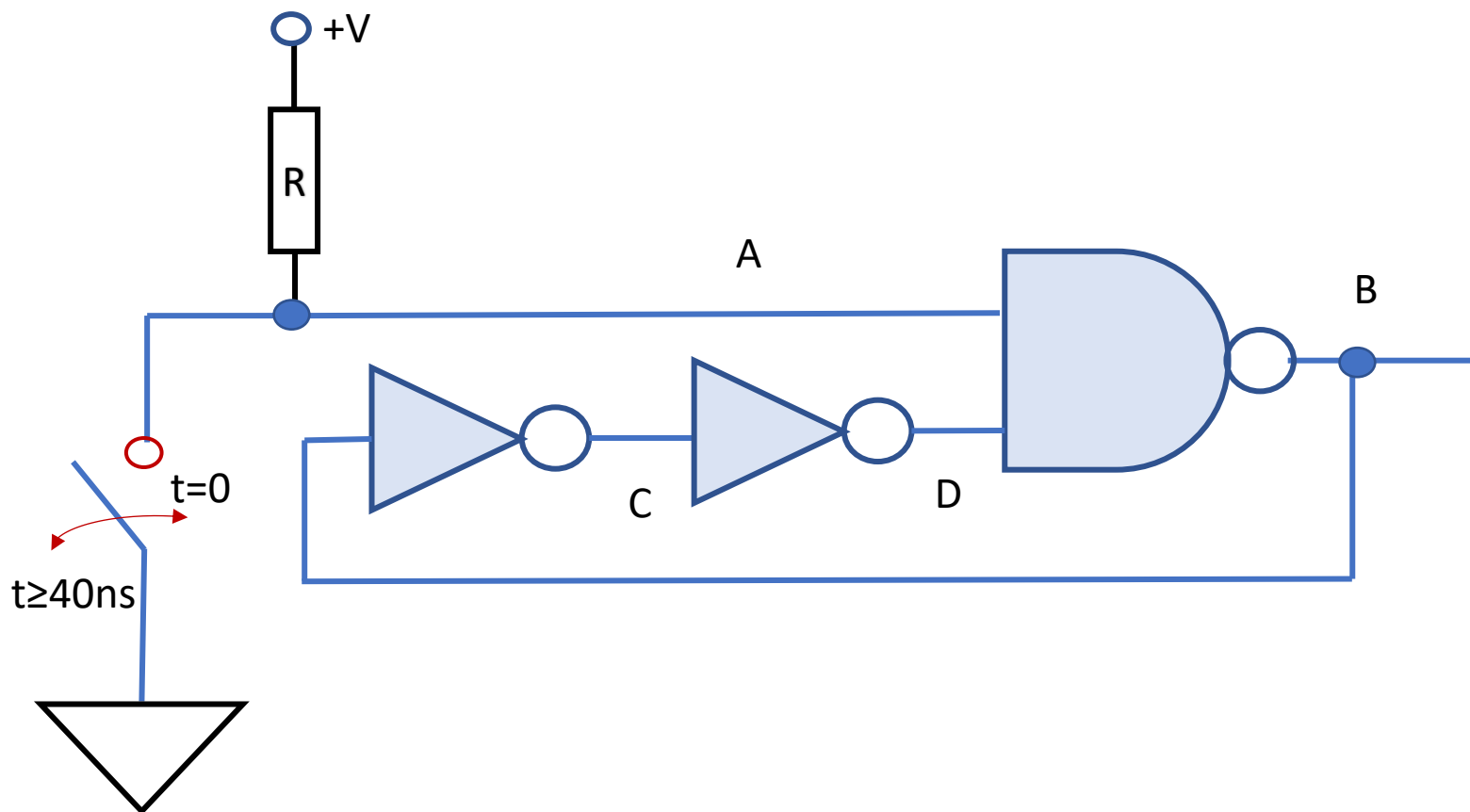
C goes to '0' at time $t=20\text{ns}$

D goes to '1' at time $t=30\text{ns}$

Since $D='1'$, $A='0'$, hence:

Output $B='0'$ always for $A='0'$ in steady state

Pulse Shaper Circuit – Case 2



Case 2:

Assume propagation delay of gates as 10ns

Output B was in steady state = '0' while Switch was closed at $t=0$

Now Switch is opened at $t=40\text{ns}$

So A goes to '1' at $t=40\text{ns}$

B goes to '0' after 10ns at $t=50\text{ns}$

C goes to '1' at $t=60\text{ns}$

D goes to '0' at $t=70\text{ns}$

Now $D='0'$ and $A='1'$, hence $B='1'$ at $t=80\text{ns}$

C goes to '0' at $t=90\text{ns}$

D goes to '1' at $t=100\text{ns}$

Since $A='1'$ and $D='1'$ so $B='0'$ at $t=110\text{ns}$

Time Period of this oscillator is determined by propagation delay of gates

Same behavior can be obtained from a NOR gate circuit with some changes

The cycle will start repeating automatically

We get an **OSCILLATOR**